12 OPERATOR OVERLOADING • THEORY

04 special operators stream and assignment

Detailed Theory, Exact C++ Rules, Memory Models & Exam Reference

04 — Overloading Stream (`<<`, `>>`), Assignment (`=`), and Subscript (`[]`) Operators

1. Overloading `<<` and `>>` for Classroom Classes

CPP

```cpp
class complex_system {
private:
    int real;
    int imag;

public:
    complex_system(int r = 0, int i = 0) : real(r), imag(i) {}

    friend ostream& operator<<(ostream& out, const complex_system& c) {
        out << c.real << " + " << c.imag << "i";
        return out; // Return ostream& to support chaining: cout << c1 << " " << c2;
    }

    friend istream& operator>>(istream& in, complex_system& c) {
        in >> c.real >> c.imag; // Note: 'c' is non-const reference so it can be modified!
        return in;
    }
};
```

Why Must `operator<<` Return `ostream&` by Reference?

1. Returning `ostream&` allows **chained output expressions** (`cout << c1 << endl;`), which groups left-to-right as `(cout << c1) << endl;` .
2. Furthermore, `std::ostream` is **non-copyable** (`std::ostream(const std::ostream&) = delete;`), so returning `ostream` by value is a compile-time error!

2. Overloading the Subscript Operator (`operator[]`)

The subscript operator `[]` must be a non-static member function and normally returns a **reference (`int&`)** so elements can be both read (`x = arr[0]`) and written (`arr[0] = 50`):

CPP

```cpp
int& operator[](int index) {
    return data[index];
}
```

3. Modern C++20 / C++23 Note: The Spaceship Operator ($\Longleftrightarrow$)

In C++20 and C++23 (`#include <compare>`), defining `auto operator⟺(const MyClass&) const = default;` automatically synthesizes all six relational comparisons ($=$, $\neq$, $<$, $\leqslant$, $>$, $\geqslant$) in a single line! In university exams, always write out explicit `bool operator==(const MyClass& other) const` unless $\Longleftrightarrow$ is specifically asked.